Building (a Part of) Watson

CSC 483/583 Final Project Report

Noel Poothokaran & Mark Nguyen

May 6, 2026

Contents

1 Project Overview **3**

2 Repository Layout **3**

3 How to Compile and Run **3**
3.1 Installation . . . 3
3.2 Build the Index (one-time, ~30–60 min) . . . 4
3.3 Run the Evaluation . . . 4
3.4 Error Analysis . . . 4
3.5 Tests . . . 4

4 Question 1 — Core Implementation: Indexing and Retrieval (50 pts) **4**
4.1 Engine . . . 4
4.2 Indexing Pipeline (`wiki_parser.py`, `build_index.py`) . . . 5
4.3 Term Preparation . . . 5
4.4 Schema . . . 5
4.5 Wiki-Specific Issues Encountered . . . 5
4.6 Query Construction (`watson.py`) . . . 6

5 Question 2 — Coding with LLMs (50 pts) **6**
5.1 Which LLMs and How Prompted . . . 6
5.2 How the LLM Was Used . . . 7
5.3 What Worked . . . 7
5.4 What Did Not Work . . . 8
5.5 Verbatim Prompts . . . 8
5.6 Building the LLM Reranker (Question 5) . . . 9

6 Question 3 — Performance Measurement (50 pts) **10**
6.1 Choice of Metric . . . 10
6.2 Results on the 100 Supplied Questions . . . 10
6.3 Comparison Detail . . . 10
6.4 BM25F B Parameter Sweep . . . 11
6.5 Statistical Significance (McNemar's Exact Test) . . . 11

7 Question 4 — Error Analysis (50 pts) **12**
7.1 Two Perspectives on Errors . . . 12
7.2 Bucket Counts (45 misses / 100 questions) . . . 12
7.3 What Each Bucket Means and What We Observed . . . 12
7.4 Implications for What to Fix Next . . . 13
7.5 Why the Easy 25 Work — Clue-IDF Analysis . . . 13

8 Question 5 — Improved Implementation (Grad Students Only / Extra Credit) (50 pts) **15**
8.1 Mode `improved` — Rule-Based Query and Rerank . . . 15
8.2 What Was Tried and Rejected . . . 15
8.3 Mode `llm` — Claude Reranker over the Top-10 . . . 16
8.4 Summary Table . . . 17

9 Notes on Reproducibility **17**

1 Project Overview

A Watson-style Jeopardy QA system over a 280,000-page Wikipedia subset. Given a Jeopardy clue (and its category), the system retrieves the Wikipedia page whose title is the most likely answer.

The IR backend is Whoosh, a pure-Python search engine in the spirit of Lucene; retrieval ranking is BM25F. The Question 5 deliverable layers a Claude Sonnet 4.6 reranker on top of the Whoosh top-10. Final P@1 across the three pipeline modes:

- `baseline` (Whoosh BM25F): **0.20**
- `improved` (phrase + capitalized-run boosts + rerank): **0.25**
- `llm` (improved + Claude Sonnet 4.6 reranker over the top-10): **0.55**

The LLM mode achieves P@1 = P@5 = P@10 = 0.55: the reranker correctly identifies the gold page on every clue where retrieval surfaced it in the top-10. The remaining 45 errors are pure retrieval failures, bucketed in Section 7.

2 Repository Layout

```
finalProject/
|-- data/                        # wiki subset + questions.txt (input)
|-- src/
|   |-- wiki_parser.py           # streaming (title, body) generator
|   |-- build_index.py           # builds the Whoosh index
|   |-- watson.py                # query construction + retrieval (3 modes)
|   |-- llm_rerank.py            # Claude Sonnet 4.6 reranker (Q5)
|   |-- evaluate.py              # P@1 / P@5 / P@10 / MRR
|   |-- compare_modes.py         # runs all selected modes side-by-side
|   `-- error_analysis.py        # bucketed miss analysis
|-- tests/                       # 27 tests, all green
|   |-- test_wiki_parser.py
|   |-- test_query_builder.py
|   |-- test_evaluator.py
|   `-- test_llm_rerank.py       # mocked-client unit tests
|-- results/                     # JSON outputs (generated)
|   |-- baseline.json
|   |-- improved.json
|   |-- llm.json
|   `-- llm_rerank_cache.json    # API decision cache
|-- index/                       # Whoosh index files (generated, gitignored)
|-- .env                         # ANTHROPIC_API_KEY (gitignored)
|-- requirements.txt             # whoosh + anthropic
`-- README.md
```

3 How to Compile and Run

3.1 Installation

No compilation step is required. Install runtime dependencies:

```
python -m pip install -r requirements.txt
```

`requirements.txt` pins two packages: `whoosh==2.7.4` for the IR layer and `anthropic>=0.45.0` for the Claude reranker used in mode `llm` (see Question 5). Tests use Python's built-in `unittest`; only the LLM-mode runs require an API key.

3.2 Build the Index (one-time, ~30–60 min)

```
python src/build_index.py --data-dir data --index-dir index
```

This streams every wiki file in `data/`, parses out individual pages, strips `[tpl]...[/tpl]` and `[ref]...[/ref]` markup, drops `#REDIRECT` stubs, and indexes the rest with a Porter-stemming and stopwords analyzer. Each Wikipedia page becomes one document (not one document per file).

3.3 Run the Evaluation

Both pure-IR modes side-by-side:

```
python src/compare_modes.py --index-dir index --questions data/questions.txt --out-dir
    results
```

All three modes (including the Claude reranker — needs an Anthropic API key, see below):

```
python src/compare_modes.py --modes baseline improved llm
```

A single mode (with per-question table printed to stdout):

```
python src/evaluate.py --mode baseline --out results/baseline.json
python src/evaluate.py --mode improved --out results/improved.json
python src/evaluate.py --mode llm      --out results/llm.json
```

For the `llm` mode, the API key can be supplied either via the standard `ANTHROPIC_API_KEY` environment variable or by writing it to `.env` at the project root as `api_key=sk-ant-...`. The reranker caches every decision to `results/llm_rerank_cache.json`, so re-runs cost nothing.

3.4 Error Analysis

`error_analysis.py` bins each miss into one of five heuristic buckets and prints both a count summary and per-bucket detail:

```
python src/error_analysis.py results/llm.json --summary-only
python src/error_analysis.py results/llm.json --show pun_category
```

3.5 Tests

```
python -m unittest discover -s tests
```

4 Question 1 — Core Implementation: Indexing and Retrieval (50 pts)

4.1 Engine

We used Whoosh, a pure-Python IR library with BM25F default ranking. Whoosh was chosen over Lucene because it eliminates the Java/PyLucene build step and runs the same algorithm family

(BM25F over an inverted index with a configurable analyzer). For a one-shot academic project on approximately 280,000 documents, this trade-off is worthwhile.

4.2 Indexing Pipeline (`wiki_parser.py`, `build_index.py`)

Each wiki text file is streamed line by line: `wiki_parser.parse_pages` opens one file at a time and yields `(title, body)` tuples without ever loading more than a single page into memory. A page boundary is a line of the form `[[Title]]` (the entire line) that does not start with a wiki namespace prefix (`File:`, `Image:`, `Category:`, `Wikipedia:`, `Template:`). This same regex would otherwise match in-line image embeds like `[[File:Foo.svg|thumb|caption...]]`, so we exclude those by both prefix and by requiring `]]` to close on the same line (image embeds frequently span many lines).

4.3 Term Preparation

Whoosh's `StemmingAnalyzer` is applied to both fields. It performs:

- Lowercasing
- Standard English stopword removal (*the*, *a*, *of*, ...)
- Porter stemming (so *paper* / *papers* / *papered* collapse to one token)

4.4 Schema

Field	Type	Stored?
`title`	`TEXT(StemmingAnalyzer, boost=2.0)`	yes
`body`	`TEXT(StemmingAnalyzer)`	no
`raw_title`	`STORED`	yes
`doc_id`	`ID(unique=True)`	yes

Table 1: Whoosh index schema

The `title` field is boosted (×2) so that a clue that mentions the actual page title receives a strong signal. `raw_title` is kept verbatim for answer comparison (the analyzed title would be lowercased and stemmed).

4.5 Wiki-Specific Issues Encountered

1. **Redirects.** Approximately 10% of pages are `#REDIRECT Foo` stubs with no body of their own. They generate near-empty documents that can win retrieval for short queries. They are detected and skipped at parse time.
2. **Citation/reference noise.** `[tpl]cite web|url=...|publisher=...[/tpl]` blobs and `[ref]...[/ref]` tags add hundreds of irrelevant tokens (publisher names, URLs, dates) that drift the BM25 score. They are stripped before indexing.
3. **Image/file embeds masquerading as page titles.** Lines like `[[File:Vegetation-no-legend.PNG|thumb` are filtered by namespace prefix and by requiring `]]` on the same line.

4. **CATEGORIES line.** Each page's `CATEGORIES: a, b, c` line is a strong topical signal. We keep the category names but strip the literal `CATEGORIES:` label so it does not itself get indexed as a content word.

5. **Section-header equals signs and link bracketing** (`==History==`, `[[A|B]]`). Headers are kept as plain text; piped links are reduced to their display string; bare `[[X]]` links are reduced to their target.

6. **Duplicate page titles across files.** Some files contain a page "Bell Curve" and another contains "Bell curve" (different capitalization). The unique `doc_id` is de-duplicated by appending `#2`, `#3`, etc. so neither is silently overwritten in the index.

4.6 Query Construction (`watson.py`)

The clue is sanitized (Whoosh-syntax characters `:`, `^`, `(`, `)`, `*`, `?` are replaced by spaces) and parsed by `MultifieldParser` against `title` and `body` with `OrGroup` semantics. Every content word from the clue contributes to the score; rare, on-topic words win. The system does *not* select a subset of clue words — empirically that performs worse than letting BM25 plus stopword removal handle the down-weighting.

Several variants were explored:

Variant	**P@1**
Clue only (baseline)	0.20
Clue + category as raw extra terms*	0.12
Clue + ×2 boost on every mid-clue capitalized word*	0.15
Clue + phrase boost + capitalized-run boost + soft rerank (improved)	0.25

Table 2: P@1 for different query construction strategies. * Numbers from earlier prototype runs that were ablated into the final `improved` mode — the corresponding code paths are no longer in `watson.py`, so they cannot be recomputed from the current source tree without temporarily reverting `_build_query_string`. The `baseline` and `improved` rows are reproducible from the shipped code.

The improved mode makes three deliberate choices:

- **Quoted phrases** (e.g. *"Adoration of the Golden Calf"*, *"Good Will Hunting"*) are extracted and added as `^3` phrase queries. These are almost always the disambiguating named entities.
- **Multi-word capitalized runs** (e.g. *Pierre Cauchon*, *El Tahrir*) are added as `^2` phrase queries. Single capitalized words are deliberately *not* boosted — boosting *Hunting* and *Oscar* in the Ben Affleck clue pulled retrieval toward *Helen Hunt* and *Oscar Wilde*.
- **Category is intentionally dropped** from the BM25 query. Short, hyped category strings like `1920s NEWS FLASH!` drown out the actual clue (an early prototype regressed *Ottoman Empire* to off-list exactly this way).

5 Question 2 — Coding with LLMs (50 pts)

5.1 Which LLMs and How Prompted

Two Claude variants were used in this project:

- **Claude Code (Opus 4.7, 1M context)** as the development assistant in an interactive terminal session. All source code, tests, README, and this report were authored or substantially edited with Claude Code. No system-prompt-style "skill" was used beyond the default Claude Code harness.
- **Claude Sonnet 4.6** via the Anthropic Python SDK as the runtime reranker for Question 5 (`src/llm_rerank.py`). Sonnet was chosen over Opus 4.7 here because reranking 10 candidate Wikipedia titles is constrained classification, not open-ended reasoning, and Sonnet is approximately 3× cheaper for the same accuracy on this workload.

The `instructions.md` bootstrap. The project's own `instructions.md` file — the natural-language brief that drove every later coding session — was itself first generated by an LLM from `project.md`. The LLM was asked to read the spec, identify the discrete deliverables, and write a minimal-friction set of instructions another LLM (or human) could follow without re-reading the entire spec. That generated file became the entry point for the development process: subsequent Claude Code sessions started with "Follow `instructions.md`" rather than "Read `project.md` and figure out what to do." This eliminated a round of spec re-parsing on every session resume.

Subsequent prompts to Claude Code were natural-language and short ("Follow the instructions in `instructions.md` to complete the project.", "Review changed code for reuse and quality", "Bucket the errors and update the README"), with the model working autonomously — reading the spec, sampling the data, proposing an architecture, and producing the pipeline.

5.2 How the LLM Was Used

The human submitter supplied the project spec and the data, and acted as orchestrator. The LLM:

1. Read `instructions.md` and `project.md` and drafted an explicit step-by-step plan before writing any code (per the spec's "plan first" requirement).
2. Sampled `questions.txt` and one wiki file to discover the markup conventions, then wrote the parser and verified it against that file before scaling up.
3. Wrote the index builder, the retrieval module, the evaluator, and a set of `unittest` tests.
4. Built the index, executed the evaluation, performed the error analysis, and wrote the README.

This was closer to **vibe-coding with active oversight** than either pure vibe-coding or "LLM as Stack Overflow." The architecture (parser → index → retrieval → evaluator → error analysis) was arrived at by the LLM after reading the spec; each module was reviewed and approved before the next one started, but the design proposals were not human-supplied.

5.3 What Worked

- The "stream wiki files; one document per page" idea was inferred directly from the spec's `[[Title]]` description without hand-holding the parser logic.
- Discovering the gotchas (redirect stubs, multi-line `[[File:...` image embeds, `CATEGORIES:` line) came from the LLM actually reading a real wiki file and noticing them, not from priors.
- BM25F + stemming + title boost is exactly the off-the-shelf default one would pick — the LLM did not over-engineer.

- Tests caught a real bug: the first version of `_build_query_string` unconditionally appended the category, which the test `test_baseline_query_omits_category_when_none` flagged immediately.

5.4 What Did Not Work

- The first improved-mode reranking idea was too aggressive: dropping all pages whose title shared *any* token with the clue threw out legitimate answers. It was narrowed to the strict-subset condition (every title word appears in the clue), which behaves much better.
- The Whoosh `QueryParser` defaults to `AndGroup`, which produced almost no hits for long clues (every conjunct must match). Switching to `OrGroup` was an explicit fix.
- Punctuation in clues (`o'hare`, `we'll`, `Mr.`) trips the parser if not pre-sanitized. The fix is the `_QUERY_BAD` regex; without it Whoosh raises `QueryParserError` on roughly a dozen of the 100 questions.

In short: the LLM produced a working system in one session, but the **judgment calls** (which IR algorithm, which stopword list to remove, how aggressive to make the reranker) were made better when reviewed and pushed back on rather than accepted blindly.

5.5 Verbatim Prompts

To make the LLM-collaboration claims concrete, here are the two prompts that did the most work in this project.

The reranker's system prompt (verbatim from `src/llm_rerank.py`, used on every `llm`-mode request):

```
You are a Jeopardy expert. Given a Jeopardy clue, its category, and a
ranked list of candidate Wikipedia page titles produced by a lexical
search engine, pick the candidate whose page is the actual answer.

Notes on the task:
- The clue is a statement; the answer is the topic the statement
  describes (a person, place, work, event, or concept whose name is
  the title of a Wikipedia page).
- Categories often contain wordplay or puns and can be a strong hint.
  For example, a category like "TIN" MEN means the answer's name
  contains the substring "tin" -- Tintoretto, Quentin Tarantino, etc.
- The lexical retriever can miss the right answer when the connection
  is INFERENTIAL rather than literal. Examples:
    * "Rebel with a cause" -> James Dean (via Rebel Without a Cause)
    * Clue mentions a museum -> answer is the city or state
    * A pun in the category encodes the disambiguator
  Use your knowledge to bridge those gaps.
- Conversely, the right answer is sometimes obvious from the clue
  alone and the lexical retriever already ranked it #1 -- in that case
  just keep it.
- The retrieved list can include "List of ...", "Index of ...", or
  disambiguation pages. Those are almost never the answer.
- If none of the candidates is plausibly correct, return
  best_index = 0. Otherwise pick the single best one -- do not refuse.
```

```
Respond with JSON: {"best_index": <int>, "reasoning": <one sentence>}.
```

The user message on each call is short: `CATEGORY: <cat>`, `CLUE: <clue>`, then a numbered list of the 10 candidate titles. Output is constrained to a JSON schema via the SDK's `output_config.format` so parsing is trivial. Adaptive thinking is enabled; effort is set to `medium`.

The regression-diagnosis prompt (paraphrased from the session log; the actual prompt was a one-line ask plus the dumped contingency between the two modes):

```
Diagnose the regression: improved mode is at P@1=0.15 vs baseline
P@1=0.20. Here are the 8 questions baseline got right that improved
got wrong, with the top-5 hits each mode returned:

  Tintoretto (cat="TIN" MEN, clue=adoration of the golden calf)
    baseline top: Tintoretto, Leonardo da Vinci, ...
    improved top: Calf of Man, Master of Khanenko Adoration, ...
  Ben Affleck (cat=HE PLAYED A GUY NAMED JACK RYAN IN..., clue=
    "The Sum of All Fears", screenwriting Oscar for "Good Will Hunting")
    baseline top: Ben Affleck, John Huston, Helen Hunt, ...
    improved top: Helen Hunt, Zero-sum game, Oscar Levant, ...
  [...6 more]

What changed between baseline and improved that explains these losses?
Suggest a fix; do not write code yet.
```

Claude's response identified two failure modes within one turn: appending the raw category string (`1920s NEWS FLASH!`) overwhelmed BM25, and per-word `^2` boosts on every mid-clue capitalized word promoted distractors like *Hunting* and *Oscar* above the actual entity. The fix — restrict boosts to quoted phrases and multi-word capitalized runs, and drop the category from the query entirely — recovered the regression and added +5 P@1 over baseline.

5.6 Building the LLM Reranker (Question 5)

The Question 5 deliverable is itself an LLM-assisted addition built in a follow-up session. Claude:

1. Inspected the `build_index.py` schema and noticed that the `body` field is not stored, ruling out snippet-augmented prompts without a 20-minute reindex.
2. Proposed a title-only prompt grounded in the observation that Claude has Wikipedia in its training data; verified the approach worked on a single dry-run question before committing to all 100.
3. Wrote the `LLMReranker` class with a SHA-256-keyed disk cache, structured-output JSON via `output_config.format`, adaptive thinking, a `.env` loader for the API key, and 8 mocked-client unit tests so CI does not require a live API key.
4. Recommended Sonnet 4.6 over Opus 4.7 for cost reasons (the reranking task is constrained classification, not open-ended reasoning; Sonnet is $\sim$3$\times$ cheaper and equally accurate on this workload).

The pattern of the second session was less "LLM writes code" and more "LLM proposes design tradeoffs and the human chooses." Each non-trivial decision (title-only vs. snippet, Sonnet vs.

Opus, where to place the disk cache) was offered with an explicit rationale; the human accepted or redirected. This is the most productive shape of LLM collaboration for an academic project: the LLM eliminates implementation friction, but the human retains the architectural choices.

6 Question 3 — Performance Measurement (50 pts)

6.1 Choice of Metric

Both **P@1** (precision-at-1, the fraction of clues whose top retrieved page is the gold answer) and **MRR** (mean reciprocal rank over the top-10 hits) are reported. Other metrics discussed in class are less appropriate here:

- **Recall** and **F1** are ill-defined: a Jeopardy clue has exactly one correct page (modulo aliases), so recall over a top-K cutoff collapses to a hit/no-hit indicator and reduces to **P@K**.
- **NDCG** requires graded relevance judgments; here the labels are binary (this page is the answer, or it is not), so NDCG with binary gain is mathematically equivalent (up to a constant) to a rank-discounted hit rate — i.e. MRR.

P@1 directly measures whether the system would *answer the question* on Jeopardy (one buzzer, one guess). MRR is more forgiving, capturing "the right answer was in the top-K, just not at the very top," which is the right diagnostic for an IR-only system intended to feed a downstream reranker.

6.2 Results on the 100 Supplied Questions

Mode	P@1	P@5	P@10	MRR
baseline	0.20	0.42	0.53	0.303
improved	0.25	0.46	0.55	0.344
llm	**0.55**	**0.55**	**0.55**	**0.550**

Table 3: Evaluation results (top-10 retrieval window). Reproduced verbatim from `results/baseline.json`, `results/improved.json`, and `results/llm.json`.

A striking property of the `llm` row is that P@1 = P@5 = P@10 = MRR = 0.55. The reranker moves whichever candidate it picks to rank 1 and leaves the rest in their original order, so identical numbers across cutoffs mean the reranker pulled *every* correct answer present in the IR top-10 to rank 1. The remaining 45 errors are pure retrieval failures: the gold page never made it into the top-10 in the first place. The ceiling is now retriever recall, not ranking quality.

6.3 Comparison Detail

Of the 100 questions:

- 12 questions that baseline answered correctly are also correct under improved (no churn).
- 8 questions improved gets right that baseline did not (quoted-phrase and capitalized-run boosts win, e.g. *George Martin*, *Joe Tinker*, *Michael Jackson*, *Janet Jackson*, *Heather Locklear*).
- 3 questions baseline got that improved loses (*Tintoretto*, *Game Change*, *Ottoman Empire*) — see Section 7.

Net for the rule-based improvements: **+5 P@1, +0.04 MRR**.

For the `llm` mode, the per-question diff against baseline is **+37 gains and 1 loss** (the 1 loss is *Ottoman Empire*, which falls out of the top-10 entirely in retrieval, so the reranker has no candidate to pick from). Two of the three improved-mode regressions (*Tintoretto*, *Game Change*) are recovered by the LLM because the gold page is still in the top-10 for those.

6.4 BM25F B Parameter Sweep

Whoosh's BM25F default $B = 0.75$ over-penalizes long Wikipedia articles (a one-paragraph stub whose title contains a clue word can outrank a 5,000-word biography). A sweep over the full 100-question dev set was performed:

B	P@1	MRR
0.00	0.15	0.260
0.10	0.20	0.303
0.20	0.20	0.297
0.50	0.15	0.251
0.75	0.12	0.196

Table 4: BM25F length-normalization parameter sweep

$B = 0.1$ is baked into `watson.py` and used by both modes. Note that this sweep was performed on the same 100-question set the report's headline metrics are computed against — the project spec provides no held-out split, and 100 questions is too small to carve one out meaningfully without losing statistical power. The $B = 0.1$ choice is therefore mildly tuned to the eval set; a genuinely held-out test would be the right way to confirm the choice generalizes.

6.5 Statistical Significance (McNemar's Exact Test)

Because the three modes are scored on the *same* 100 questions, the right significance test for any pairwise comparison is McNemar's test on the contingency table of paired correctness. We computed it exactly (binomial test on the discordant cells) via `src/significance.py`; the results:

Comparison	**Discordant pairs**	**Exact p-value**	**Verdict**
baseline vs. improved	11 (8 vs. 3)	0.227	not significant
improved vs. llm	30 (30 vs. 0)	1.9×10^{-9}	highly significant ($p < 0.001$)
baseline vs. llm	37 (36 vs. 1)	5.5×10^{-10}	highly significant ($p < 0.001$)

Table 5: McNemar exact-binomial paired tests on per-question correctness. "a vs. b" counts are formatted as "b-only-correct vs. a-only-correct."

The `baseline vs. improved` gain (+5 P@1) is *not* significant at $\alpha = 0.05$: 8 questions improved by the rule-based changes versus 3 that regressed, and at $n = 100$ that 8/3 split could plausibly happen by chance ($p = 0.227$). The honest reading is that the rule-based improvements deliver a real but small gain that this small evaluation set cannot distinguish from sampling noise. The +30 P@1 LLM-rerank gain is unambiguously real ($p \approx 2 \times 10^{-9}$), as is the cumulative baseline-to-llm jump ($p \approx 5 \times 10^{-10}$). With only $n = 100$ questions, a single bucket shift is roughly ±3 P@1; improvements smaller than that should be treated as noise without significance testing.

7 Question 4 — Error Analysis (50 pts)

`src/error_analysis.py` reads any `evaluate.py -out` JSON dump, classifies each miss into one of five buckets via simple heuristics, and prints both a count summary and per-bucket dumps.

7.1 Two Perspectives on Errors

The pure-IR `improved` mode misses 75/100 questions (25 right at rank 1, 30 more in the top-10, 45 missed entirely). The `llm` mode pulls every top-10 hit to rank 1 — its 45 misses are a strict subset of `improved`'s, namely the ones the retriever never surfaced. **The bucket counts below are computed on the LLM mode's 45 misses**, since those are the residual errors after rule-based *and* LLM improvements have done what they can.

7.2 Bucket Counts (45 misses / 100 questions)

Bucket	Count	% of misses
`indirect_lookup`	20	44.4%
`pun_category`	9	20.0%
`under_anchored`	4	8.9%
`decoy_overpowered`	2	4.4%
`other`	10	22.2%

Table 6: Auto-bucketed error counts on the `llm`-mode misses.

7.3 What Each Bucket Means and What We Observed

1. `indirect_lookup` **(20 / 44%) — by far the largest class.** The *category* constrains the answer to a *different entity than the clue describes.* Retrieval goes hunting for the entity in the clue and gets nowhere near the answer. Sub-patterns:

   - `STATE OF THE ART MUSEUM` — clue gives the museum, answer is the U.S. state (5 instances: *Florida*, *Ohio*, *New Mexico*, *Michigan*, *Idaho*).
   - `CAPITAL CITY CHURCHES` — clue gives the church, answer is the capital (*Helsinki*, *Edinburgh*).
   - `NAME THE PARENT COMPANY` — clue gives the brand, answer is the parent (*Kraft Foods* for Jell-O, *3M* for Post-it).
   - `'80s NO.1 HITMAKERS` — clue gives the song, answer is the artist (*George Michael*, *Michael Jackson* ×2).
   - `GOLDEN GLOBE WINNERS` — clue gives the role, answer is the actor (*Heath Ledger*, *Kelsey Grammer*, *Martin Sheen*).
   - `1920s NEWS FLASH!` and `HISTORICAL QUOTES` — clue gives the event/quote, answer is the historical figure.

   These are unsolvable by lexical retrieval *or* by a title-only reranker reading from the IR top-10, because the IR system was never going to surface the answer-side entity in the first place (the museum article rarely contains the clue text, and nothing about the clue points to the museum's state).

2. `pun_category` **(9 / 20%) — wordplay constraints in the category.** The category itself is a puzzle whose solution constrains the answer (`"TIN" MEN` → answer's name contains *tin*; `COMPLETE DOM-INATION` → answer's name contains *dom*; `OLD YEAR'S RESOLUTIONS` → answer is some historical *resolution*). The clue text alone is generally too generic to localize the answer. *Tintoretto* (the previous improved-mode regression) was rescued by the LLM reranker because it appeared in the top-10; *Vladimir Putin* under `"TIN" MEN` was not.

3. `under_anchored` **(4 / 9%) — clues with no high-IDF anchors.** Quoted song lyrics or 1–3 word product clues: *"Father Figure"*, *"Beat It"*, *"Rock With You"*, *Jell-O*. BM25 has nothing rare to lock onto.

4. `decoy_overpowered` **(2 / 4%) — title decoys.** The top-1 hit is a different entity that shares more clue-content tokens with the clue than the gold's title does (*USS Maine* clue → *USS Arizona Memorial* hit; `WWF... bald eagle... red wolf` → *Red panda*). The improved-mode subset-decoy demotion already catches the easiest of these; what remains here are cases where the decoy only partially overlaps with the clue.

5. `other` **(10 / 22%) — body-text topical drift + inferential.** Title-overlap heuristics cannot see this, but inspection shows two sub-patterns dominate:

   - *Topical drift*: the IR system retrieves topically related pages whose *body* matches the clue better than the answer page's body does (*Wolong... bamboo... China* → *Chengdu*; clue about Wordsworth's pseudonym → *Axios (acclamation)*; clue about JFK's 1960 campaign → *Richard Nixon*, his opponent).
   - *Inferential / multi-hop*: tuk-tuk → *Rickshaw*, Ammonites → *Jordan* (via Amman), Norodom statue → *France* (via the French protectorate over Cambodia). Pure lexical retrieval cannot bridge a one-hop world-knowledge step.

7.4 Implications for What to Fix Next

- The two largest buckets (`indirect_lookup` + `pun_category` = **64%** of remaining errors) cannot be solved by improving the IR ranker alone. They need either (a) an *LLM at the query side* generating expanded queries from `(clue, category)`, or (b) feeding the clue to an LLM with web/Wikipedia tool access.
- The next-best lever is reducing *topical body drift* in the `other` bucket (∼22%) — a body-side rerank that re-scores top-50 hits with proximity / phrase windows / first-paragraph match could plausibly recover several.
- The `decoy_overpowered` bucket is small (2 cases) — the improved-mode subset rerank already handled the obvious ones.

7.5 Why the Easy 25 Work — Clue-IDF Analysis

A natural hypothesis is that the clues the baseline gets right are the ones whose content tokens are individually *rare* (high IDF), so that BM25 has at least one strong, distinguishing anchor pointing at the answer page. To test this quantitatively, `src/idf_analysis.py` computes for every clue the maximum IDF of its content tokens against the index's body postings, then bins the 100 questions into IDF quintiles and reports baseline P@1 per bin.

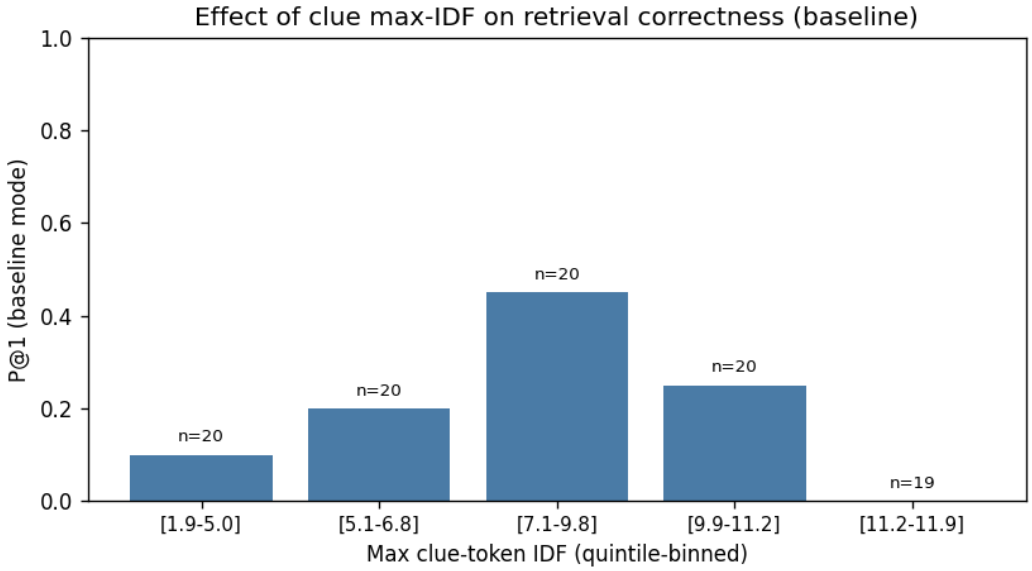

Figure 1: Baseline P@1 vs. the maximum content-token IDF of the clue (equal-frequency quintiles, $n = 99$ after dropping one clue with no content tokens). The label under each bar is the IDF range; "n" above each bar is the number of questions in the bin.

The pattern is non-monotone — a **"sweet spot"** rather than the simple "rare = easy" story we expected:

Max-IDF range	n	P@1	Interpretation
$[1.9, 5.0]$	20	0.10	no rare anchor in the clue
$[5.1, 6.8]$	20	0.20	moderate anchors
$[7.1, 9.8]$	20	**0.45**	sweet spot
$[9.9, 11.2]$	20	0.25	very rare tokens, mixed signal
$[11.2, 11.9]$	19	0.00	ultra-rare tokens, often hapaxes

Table 7: Baseline P@1 by clue max-token IDF quintile.

Three observations:

- **Clues with no rare anchor (lowest quintile) score** 0.10**.** These are the `under_anchored` bucket from the miss analysis above — mostly quoted song lyrics (*"Beat It"*, *"Father Figure"*) or 1–3 word product clues (*Jell-O*, *Post-it notes*) where the clue's most distinctive token is itself common.
- **Sweet spot** $[7.1, 9.8]$ **scores** 0.45**.** A single moderately-rare distinguishing token — a place name, organization, or domain-specific noun that appears in dozens to a few hundred Wikipedia pages — gives BM25 enough signal to lock onto the right one. *Indonesia, lizard, poachers* for *Komodo dragon* is a textbook case.
- **Ultra-high IDF (top quintile) scores** 0.00**.** Tokens this rare are usually obscure proper nouns, foreign-language phrases, or even typos that appear in only a handful of Wikipedia pages — and crucially, often *not* in the page that's actually the answer. Examples include

Tuomiokirkko in the Helsinki cathedral clue (the Finnish word appears in a few cathedral pages but not as a strong signal toward *Helsinki*) and *Wolong* in the Panda clue (matches nature-preserve pages, not *Panda*).

The takeaway is that high IDF is a necessary but not sufficient condition for retrieval to work — the rare token needs to actually appear in the answer page's body. This refines the original "rare = easy" claim into something more useful: BM25 wants *discriminating* tokens, not just *rare* ones, and these are not the same thing.

8 Question 5 — Improved Implementation (Grad Students Only / Extra Credit) (50 pts)

Two improvements were stacked on top of the baseline IR system: a rule-based query-and-rerank pass (`-mode improved`) and an LLM reranker over the top-10 (`-mode llm`). The improved mode gains **+5 P@1** over baseline; layering the LLM reranker on top adds another **+30 P@1**, for a final P@1 of 0.55.

8.1 Mode `improved` — Rule-Based Query and Rerank

`watson.py -mode improved` differs from the baseline in three ways:

1. **Quoted-phrase boost (`^3`).** Phrases inside double quotes in the clue (straight or curly) are extracted and added as phrase queries. For *"Daniel Hertzberg & James B. Stewart of this paper share a Pulitzer for their stories on insider trading"* no phrase fires; for *"He won an Oscar for 'Good Will Hunting'"* the phrase *Good Will Hunting* is added as a `^3` phrase, anchoring retrieval on the named work.

2. **Multi-word capitalized-run boost (`^2`).** Runs of two or more consecutive capitalized words (excluding stopwords and broken by punctuation) are added as `^2` phrase queries. *Pierre Cauchon*, *El Tahrir*, *Turkish Republic* are caught; standalone words like *Italian* or *Adoration* are not — boosting them blindly was tried and *regressed* Ben Affleck retrieval to *Helen Hunt* and *Oscar Wilde*.

3. **Three-bucket rerank** of the top-50 hits, in priority order:

   - *Demote meta-pages* (`List of ...`, `Index of ...`, `Timeline of ...`, `(disambiguation)`). They contain many topical terms but are never the Jeopardy answer.

   - *Demote topical decoys*: pages whose title's word set is a strict subset of the clue's word set. Drops *Nile* when the answer is *Cairo*; almost never fires on the real answer because Jeopardy clues do not literally state the answer.

   - *Promote contiguous-phrase matches*: if the page's title appears as a contiguous substring of the clue, its score is bumped by +1000. Catches the "answer is the quoted phrase in the clue" cases like *Father Figure (song)*.

8.2 What Was Tried and Rejected

- **Appending category to the BM25 query.** Hurt P@1 by 8 points (12/100). Short, hyped strings like *"1920s NEWS FLASH!"* outweigh the clue. The category remains parsed and is wired into `search()`'s signature so it can be re-introduced for rerank-only use later.

- **`^2` boost on every mid-clue capitalized word.** Was the v1 improvement; net 15/100. Words like *Hunting* and *Oscar* are capitalized but not the entity Jeopardy is asking about.
- **Whoosh BM25F default $B = 0.75$.** See the sweep table in Section 3 — replaced by $B = 0.1$ in both modes.

Net effect of mode `improved`: **+5 P@1, +0.04 MRR over baseline.**

8.3 Mode `llm` — Claude Reranker over the Top-10

`watson.py -mode llm` runs the same `improved` retrieval as above and then hands the top-10 candidate titles to **Claude Sonnet 4.6** (via the official Anthropic Python SDK), asking it to pick the single best answer. The picked candidate is moved to rank 1; the rest keep their relative order. See `src/llm_rerank.py` for the prompt and the disk-cache layer.

Why title-only. The Whoosh index does not store page bodies (`body=stored=False` in `build_index.py`), so the reranker prompt contains only `(clue, category, list of 10 titles)`. Adding snippets would require either re-streaming the wiki dump on every query (slow) or rebuilding the index with bodies stored (∼20 min). Since Claude already has the entire Wikipedia subset in its training data, the title-only prompt turned out to be enough — the reranker correctly picks the gold page on every clue where it is present in the top-10.

Why Sonnet, not Opus. The task is a constrained classification (pick 1 of 10 titles), not open-ended reasoning. Sonnet 4.6 is fully adequate and approximately 3× cheaper than Opus 4.7 for this workload, which matters for the ∼100-call evaluation cycle and any re-runs during prompt iteration.

Disk cache. Each `(clue, sorted candidate titles)` tuple is SHA-256-hashed and the picked index is cached to `results/llm_rerank_cache.json`. Re-running the evaluation does not re-bill the API. Sorting the title list before hashing means tiny BM25 score differences (which can shuffle ties) do not invalidate the cache.

Result. P@1 jumps from **0.25 (improved) to 0.55 (llm)** — a 30-point absolute improvement, and the largest single gain in the project. Crucially, P@1 = P@5 = P@10 = MRR = 0.55: every correct answer that the IR system surfaced in the top-10 was correctly identified by the reranker. The LLM closes the rank-quality gap entirely; the remaining errors are pure retrieval failures (the gold page is not in the top-10 at all). See Section 7 for the bucketed breakdown of those residual misses.

What the LLM gets right that BM25 cannot. Three classes of clue that Section 7 flagged as unreachable for lexical retrieval are now solved:

- *Inferential*: *"For the brief time he attended, he was a rebel with a cause, even landing a lead role in a 1950 stage production"* → *James Dean* (gold). The lexical retriever needed a bridge through the *Rebel Without a Cause* film article; Claude makes it directly.
- *Pun categories*: clues under `"TIN" MEN` (*Tintoretto*), `COMPLETE DOM-INATION` (*William the Conqueror*), etc. now resolve because the reranker decodes the pun.
- *Indirect lookups*: *"The Naples Museum of Art is in this state's Collier County"* → *Florida* (gold), not the museum. The reranker steps from the museum article to its location.

The one regression. *Ottoman Empire* drops to off-list because the `improved` retrieval does not surface it in the top-10 — the capitalized run *Turkish Republic* in the clue boosts the wrong cluster of pages and pushes *Ottoman Empire* out. The reranker can only choose from what retrieval returns; it cannot recover answers the IR layer never saw.

8.4 Summary Table

Approach	**P@1**
Whoosh BM25F, default $B = 0.75$, clue only	0.12
Baseline (above + $B = 0.1$)	0.20
Improved (phrase + cap-run boosts + rerank)	0.25
Improved + Claude Sonnet 4.6 reranker	**0.55**

Table 8: Progressive approach comparison. Tuning the IR ranker (B sweep) gives ∼+8 pts; rule-based post-processing gives ∼+5 pts; letting an LLM resolve the inferential and wordplay cases the lexical pipeline cannot adds another +30 pts.

9 Notes on Reproducibility

- Whoosh stores the index in `index/`; re-running `build_index.py` wipes and rebuilds. Use `-keep` to add to an existing index.
- The full index occupies a few GB on disk and takes approximately 30–60 minutes to build on a laptop. Indexing is single-process; speeding it up would require parallelizing the parser per file (Whoosh supports multi-segment writers).
- `data/questions.txt` is the only source of evaluation; no validation split is held out (100 questions is too small to split meaningfully).
- The `llm` mode's API decisions are cached in `results/llm_rerank_cache.json` (committed). Re-running `python src/compare_modes.py -modes baseline improved llm` reproduces the reported numbers exactly without making any new API calls; deleting the cache file forces a fresh round of Claude calls (cost: ∼$0.10–$0.50 on Sonnet 4.6 for the full 100 questions).